

SOLID 检查清单

1. 实验说明

本清单按 P3 要求，先假设 AI 给出了一版“可运行但不够工程化”的初始设计，再对照当前项目结构进行审查和修正。修正目标是让设计更贴近当前后端模块划分：

- controller
- service
- entity
- mapper
- config
- util

2. AI 原始设计摘要

AI 初稿曾给出以下倾向性问题：

1. 用一个 PlatformService 同时处理用户、任务、评论、评价、消息。
2. 让 Task 实体直接包含接单、完成、发消息、评价结算等全部行为。
3. 控制器直接依赖数据库访问层，未经过服务层封装。
4. 外部实名认证和内容审核写死在业务类内部。
5. 接口只按“功能可用”设计，缺乏扩展点和隔离层。

3. 检查表

SOLID 原则	检查问题	AI 设计是否违反	违反说明	修正方案
S - 单一职责	有没有类承担了过多职责？	是	PlatformService 同时承担认证、发任务、接单、发消息、评价结算、后台审核，职责过多，修改原因高度耦合。	拆为 AuthService、UserService、TaskService、TaskCommentService、TaskReviewService、MessageService、FeedbackService、AdminService。

O - 开闭原则	新增需求类型是否需要修改现有代码?	是	AI 初稿把任务类型判断写死在一个大分支里, 新增“树洞/搭子/咨询”时必须修改原逻辑。	引入 taskMode 概念和策略化处理, 新增模式时增加新策略或新规则对象。
L - 里氏替换	子类是否可以替换父类使用?	部分违反	AI 曾把“匿名话题”继承自“普通任务”, 但匿名话题并不支持接单和完成, 继承语义不成立。	不用继承表达业务差异, 改为 Task + taskMode + 行为策略。
I - 接口隔离	有没有接口太胖, 包含了不需要的方法?	是	初稿存在 TaskManageService, 同时暴露发布、接单、评论、评价、审核、消息等方法, 调用方只需一部分能力。	按业务边界拆成小接口; 评论、评价、消息、审核独立为专门服务。
D - 依赖倒转	高层模块是否直接依赖了低层模块的具体实现?	是	业务逻辑直接依赖第三方校园认证、审核 SDK, 且控制器直接操作 Mapper。	控制器依赖 Service; 外部能力抽象为适配接口, 由基础设施层提供实现。

4. 发现的问题数量与修正

- 明显违反 SOLID 的设计问题: 5 处
- 其中影响最大的 2 处:
 - Task 领域过胖, 导致任务、话题、评论、评价耦合
 - 控制器越过服务层直接访问数据层, 破坏分层边界

5. 修正后的设计判断

5.1 单一职责

当前项目的控制器和服务命名已经体现按业务域拆分, 是正确方向。后续应避免再把“任务状态机”“消息通知”“积分结算”重新塞回同一个服务。

5.2 开闭原则

taskMode 已经是很关键的扩展点。后续如果新增活动报名、失物招领、学习小组等形态, 建议扩展策略, 不直接改已有核心流程。

5.3 里氏替换

该系统更适合“组合 + 状态/模式”而不是“深继承树”。因为任务、话题、反馈等对象共享部分字段，但行为差异大。

5.4 接口隔离

接口应按调用方划分：

- 学生端：任务、评论、评价、消息
- 管理端：审核、封禁、公告、仲裁
- 外部依赖：认证、审核、支付适配

5.5 依赖倒转

高层模块应只依赖抽象：

- 认证依赖 `AuthProvider`
- 内容审核依赖 `ContentModerationGateway`
- 支付依赖 `PaymentGateway`

6. 结论

本次 SOLID 审查表明，AI 能快速生成“像样”的初稿，但在以下方面仍容易失真：

- 把功能堆在少数大类中
- 用继承误表达业务差异
- 低估第三方接入隔离的重要性
- 忽略后台审核与审计链路

因此，AI 适合提供结构草稿，不适合替代最终的模块边界判断。